

尚硅谷大数据项目之电商数仓（3 数仓数据同步策略）

（作者：尚硅谷研究院）

版本：V5.0

第 1 章 实时数仓同步数据

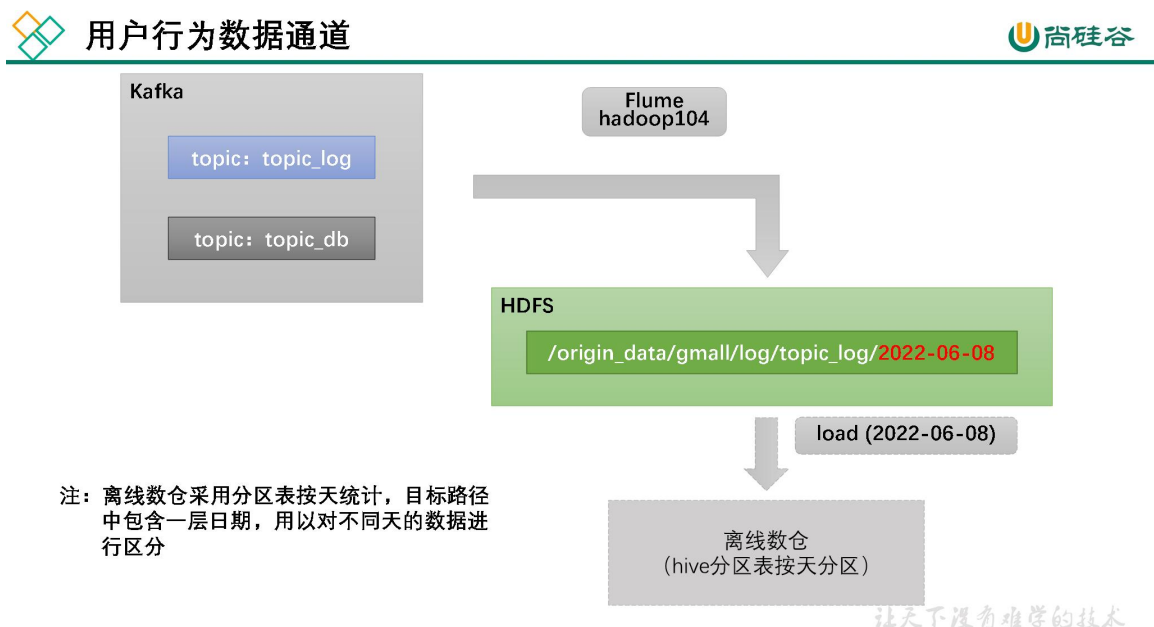
实时数仓由 Flink 源源不断从 Kafka 当中读数据计算，所以不需要手动同步数据到实时数仓。

第 2 章 离线数仓同步数据

2.1 用户行为数据同步

2.1.1 数据通道

用户行为数据由 Flume 从 Kafka 直接同步到 HDFS，由于离线数仓采用 Hive 的分区表按天统计，所以目标路径要包含一层日期。具体数据流向如下图所示。



2.1.2 日志消费 Flume 配置概述

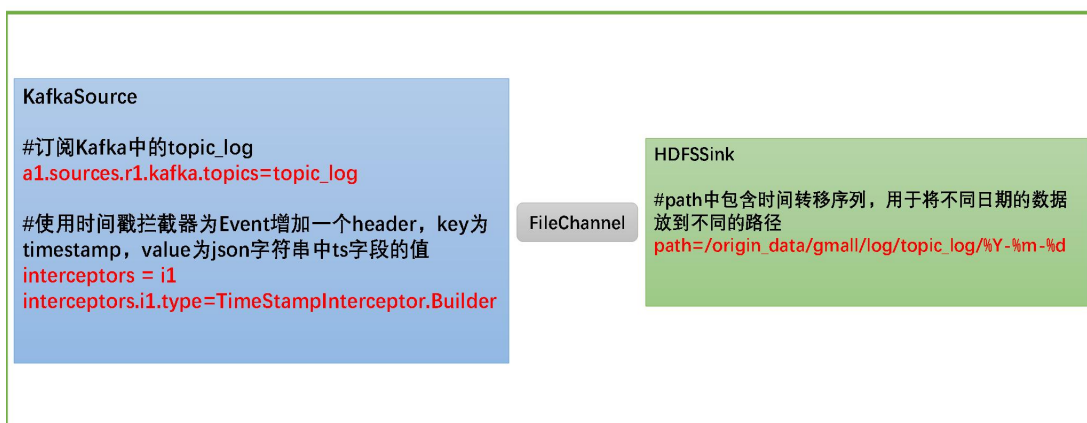
按照规划, 该 Flume 需将 Kafka 中 topic_log 的数据发往 HDFS。并且对每天产生的用户行为日志进行区分, 将不同天的数据发往 HDFS 不同天的路径。

此处选择 KafkaSource、FileChannel、HDFS Sink。

关键配置如下:



日志消费 Flume 关键配置



让天下没有难学的技术

2.1.3 日志消费 Flume 配置实操

1) 创建 Flume 配置文件

在 hadoop104 节点的 Flume 家目录下创建 job 目录, 在 job 下创建 kafka_to_hdfs_log.conf

```
[atguigu@hadoop104 ~]$ cd /opt/module/flume/
[atguigu@hadoop104 flume]$ mkdir job
[atguigu@hadoop104 flume]$ vim job/kafka_to_hdfs_log.conf
```

2) 配置文件内容如下

```
#定义组件
a1.sources=r1
a1.channels=c1
a1.sinks=k1

#配置 source1
a1.sources.r1.type = org.apache.flume.source.kafka.KafkaSource
a1.sources.r1.batchSize = 5000
a1.sources.r1.batchDurationMillis = 2000
a1.sources.r1.kafka.bootstrap.servers = hadoop102:9092,hadoop103:9092,hadoop104:9092
a1.sources.r1.kafka.topics=topic_log
a1.sources.r1.kafka.consumer.group.id=flume_log1

#拦截器
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载, 可百度访问: [尚硅谷官网](#)

```
a1.sources.r1.interceptors = i1
a1.sources.r1.interceptors.i1.type =
com.atguigu.gmall.flume.interceptor.TimestampInterceptor$Builder

#配置 channel
a1.channels.c1.type = file
a1.channels.c1.dataDirs = /opt/module/flume-1.10.1/data/behavior1
a1.channels.c1.checkpointDir =
/opt/module/flume-1.10.1/checkpoint/behavior1
a1.channels.c1.maxFileSize = 2146435071
a1.channels.c1.capacity = 1000000
a1.channels.c1.keep-alive = 6

#配置 sink
a1.sinks.k1.type = hdfs
a1.sinks.k1.hdfs.path = /origin_data/gmall/log/topic_log/%Y-%m-%d
a1.sinks.k1.hdfs.filePrefix = log
a1.sinks.k1.hdfs.round = false

a1.sinks.k1.hdfs.rollInterval = 10
a1.sinks.k1.hdfs.rollSize = 134217728
a1.sinks.k1.hdfs.rollCount = 0

#控制输出文件类型
a1.sinks.k1.hdfs.fileType = CompressedStream
a1.sinks.k1.hdfs.codeC = gzip

#组装
a1.sources.r1.channels = c1
a1.sinks.k1.channel = c1
```

3) FileChannel 优化

通过配置 **dataDirs** 指向多个路径，每个路径对应不同的硬盘，增大 Flume 吞吐量。

官方说明如下：

```
Comma separated list of directories for storing log files. Using
multiple directories on separate disks can improve file channel
performance
```

checkpointDir 和 **backupCheckpointDir** 也尽量配置在不同硬盘对应的目录中，保证 checkpoint 坏掉后，可以快速使用 **backupCheckpointDir** 恢复数据

4) HDFS Sink 优化

(1) HDFS 存入大量小文件，有什么影响？

元数据层面：每个小文件都有一份元数据，其中包括文件路径，文件名，所有者，所属组，权限，创建时间等，这些信息都保存在 Namenode 内存中。所以小文件过多，会占用 Namenode 服务器大量内存，影响 Namenode 性能和使用寿命

计算层面：默认情况下 MR 会对每个小文件启用一个 Map 任务计算，非常影响计算性能。同时也影响磁盘寻址时间。

(2) HDFS 小文件处理

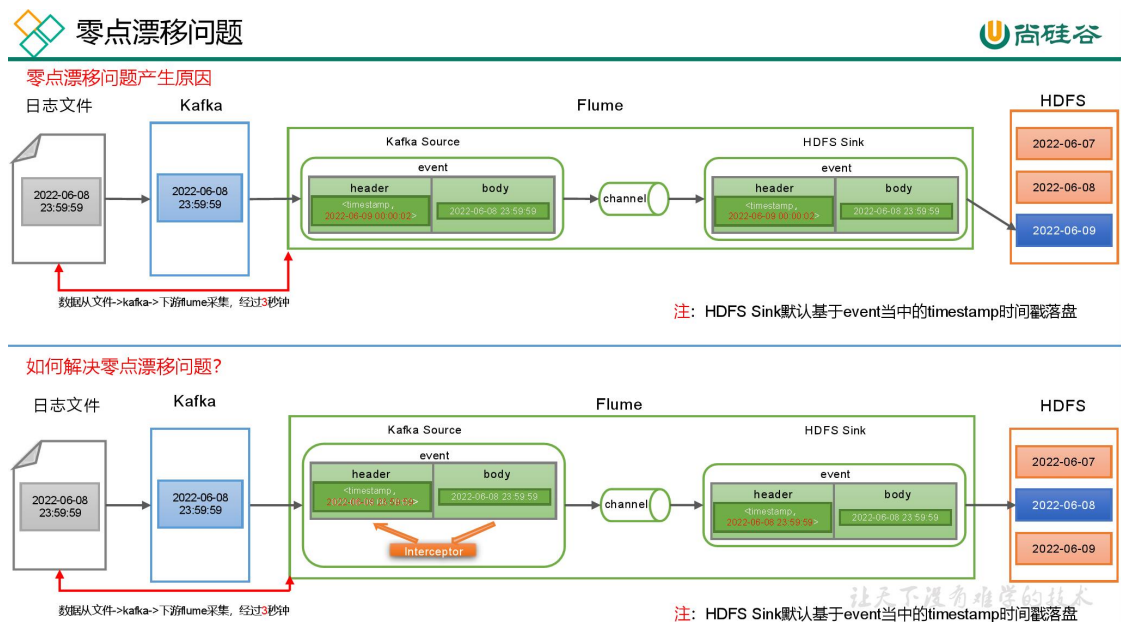
官方默认的这三个参数配置写入 HDFS 后会产生小文件，hdfs.rollInterval、hdfs.rollSize、hdfs.rollCount

基于以上 hdfs.rollInterval=3600，hdfs.rollSize=134217728，hdfs.rollCount=0 几个参数综合作用，效果如下：

- 文件在达到 128M 时会滚动生成新文件
- 文件创建超 3600 秒时会滚动生成新文件

5) 编写 Flume 拦截器

(1) 零点漂移问题



(2) 在 idea 里创建名为 gmall 的项目

Name:

Location:

Project will be created in: D:\coding\gmall

☐ Create Git repository

Language:

Java
Kotlin
Groovy
Scala
JavaScript
+

Build system:

IntelliJ
Maven
Gradle

JDK:

1.8 java version "1.8.0_172"

☒ Add sample code

Advanced Settings

GroupId:

ArtifactId:

(3) 在 pom.xml 文件中添加如下配置

```

<dependencies>
  <dependency>
    <groupId>org.apache.flume</groupId>
    <artifactId>flume-ng-core</artifactId>
    <version>1.10.1</version>
    <scope>provided</scope>
  </dependency>

  <dependency>
    <groupId>com.alibaba</groupId>
    <artifactId>fastjson</artifactId>
    <version>1.2.62</version>
  </dependency>
</dependencies>

<build>
  <plugins>
    <plugin>
      <artifactId>maven-compiler-plugin</artifactId>
      <version>2.3.2</version>
      <configuration>
        <source>1.8</source>
        <target>1.8</target>
      </configuration>
    </plugin>
    <plugin>
      <artifactId>maven-assembly-plugin</artifactId>
      <configuration>
        <descriptorRefs>

```

```
<descriptorRef>jar-with-dependencies</descriptorRef>
  </descriptorRefs>
</configuration>
<executions>
  <execution>
    <id>make-assembly</id>
    <phase>package</phase>
    <goals>
      <goal>single</goal>
    </goals>
  </execution>
</executions>
</plugin>
</plugins>
</build>
```

(4) 在 `com.atguigu.gmall.flume.interceptor` 包下创建 `TimestampInterceptor` 类

```
package com.atguigu.gmall.flume.interceptor;

import com.alibaba.fastjson.JSONObject;
import org.apache.flume.Context;
import org.apache.flume.Event;
import org.apache.flume.interceptor.Interceptor;
import java.nio.charset.StandardCharsets;
import java.util.Iterator;

import java.util.List;
import java.util.Map;

public class TimestampInterceptor implements Interceptor {

    @Override
    public void initialize() {

    }

    @Override
    public Event intercept(Event event) {
        //1、获取 header 和 body 的数据
        Map<String, String> headers = event.getHeaders();
        String log = new String(event.getBody(), StandardCharsets.UTF_8);

        try {
            //2、将 body 的数据类型转成 jsonObject 类型（方便获取数据）
            JSONObject jsonObject = JSONObject.parseObject(log);

            //3、header 中 timestamp 时间字段替换成日志生成的时间戳（解决数据漂移问题）
            String ts = jsonObject.getString("ts");
            headers.put("timestamp", ts);

            return event;
        } catch (Exception e) {
            e.printStackTrace();
            return null;
        }
    }
}
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

```

}

@Override
public List<Event> intercept(List<Event> list) {
    Iterator<Event> iterator = list.iterator();
    while (iterator.hasNext()) {
        Event event = iterator.next();
        if (intercept(event) == null) {
            iterator.remove();
        }
    }
    return list;
}

@Override
public void close() {

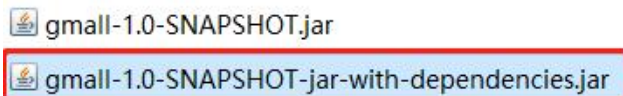
}

public static class Builder implements Interceptor.Builder {
    @Override
    public Interceptor build() {
        return new TimestampInterceptor();
    }

    @Override
    public void configure(Context context) {
    }
}
}

```

(5) 打包



(6) 需要先将打好的包放入到 hadoop104 的/opt/module/flume/lib 文件夹下面。

2.1.4 日志消费 Flume 测试

1) 启动 Zookeeper、Kafka、HDFS

2) 启动日志采集 Flume

```
[atguigu@hadoop102 ~]$ fl.sh start
```

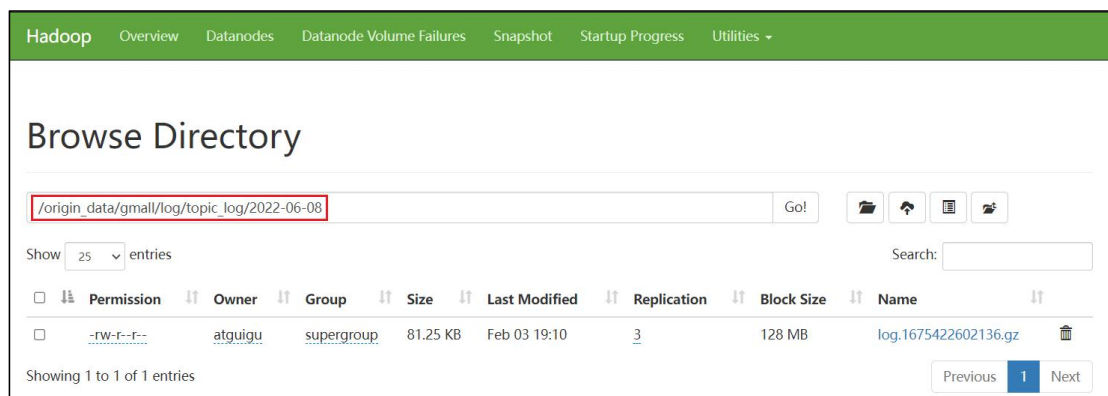
3) 启动 hadoop104 的日志消费 Flume

```
[atguigu@hadoop104 flume]$ bin/flume-ng agent -n a1 -c conf/ -f
job/kafka_to_hdfs_log.conf
```

4) 生成模拟数据

```
[atguigu@hadoop102 ~]$ lg.sh
```

5) 观察 HDFS 是否出现数据



2.1.5 日志消费 Flume 启停脚本

若上述测试通过，为方便，此处创建一个 Flume 的启停脚本。

1) 在 hadoop102 节点的/home/atguigu/bin 目录下创建脚本 f2.sh

```
[atguigu@hadoop102 bin]$ vim f2.sh
```

在脚本中填写如下内容。

```
#!/bin/bash

case $1 in
"start")
    echo " -----启动 hadoop104 日志数据 flume-----"
    ssh hadoop104 "nohup /opt/module/flume-1.10.1/bin/flume-ng
agent -n a1 -c /opt/module/flume-1.10.1/conf -f
/opt/module/flume-1.10.1/job/kafka_to_hdfs_log.conf >/dev/null 2>&1
&"
    ;;
"stop")
    echo " -----停止 hadoop104 日志数据 flume-----"
    ssh hadoop104 "ps -ef | grep kafka_to_hdfs_log | grep -v grep
| awk '{print \$2}' | xargs -n1 kill"
    ;;
esac
```

2) 增加脚本执行权限

```
[atguigu@hadoop102 bin]$ chmod 777 f2.sh
```

3) f2 启动

```
[atguigu@hadoop102 module]$ f2.sh start
```

4) f2 停止

```
[atguigu@hadoop102 module]$ f2.sh stop
```

2.2 业务数据同步

2.2.1 数据同步策略概述

业务数据是数据仓库的重要数据来源，我们需要每日定时从业务数据库中抽取数据，传

输到数据仓库中，之后再对数据进行分析统计。

为保证统计结果的正确性，需要保证数据仓库中的数据与业务数据库是同步的，离线数仓的计算周期通常为天，所以数据同步周期也通常为天，即每天同步一次即可。

数据的同步策略有**全量同步**和**增量同步**。

全量同步，就是每天都将业务数据库中的全部数据同步一份到数据仓库，这是保证两侧数据同步的最简单的方式。



全量同步

日期: 2022-06-10

业务数据库

id	name
1	张三
2	李小四
3	王五
4	赵六
5	田七

数据仓库

2022-06-08		2022-06-09		2022-06-10	
id	name	id	name	id	name
1	张三	1	张三	1	张三
2	李四	2	李小四	2	李小四
		3	王五	3	王五
				4	赵六
				5	田七

让天下没有难学的技术

增量同步，就是每天只将业务数据中的新增及变化数据同步到数据仓库。采用每日增量同步的表，通常需要在首日先进行一次全量同步。

日期：2022-06-10

业务数据库

id	name
1	张三
2	李小四
3	王五
4	赵六
5	田七

数据仓库

2022-06-08		2022-06-09		2022-06-10	
id	name	id	name	id	name
1	张三	2	李小四	4	赵六
2	李四	3	王五	5	田七

让天下没有难学的技术

2.2.2 数据同步策略选择

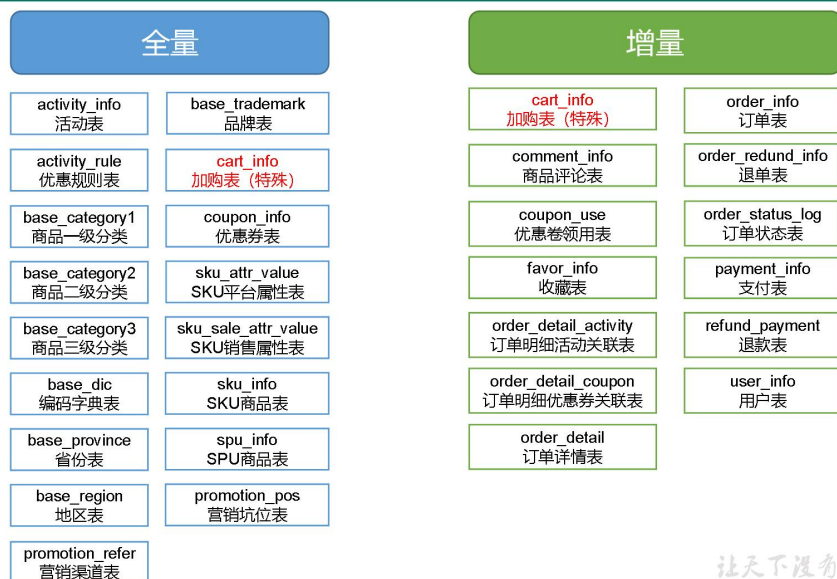
两种策略都能保证数据仓库和业务数据库的数据同步，那应该如何选择呢？下面对两种策略进行简要对比。

同步策略	优点	缺点
全量同步	逻辑简单	在某些情况下效率较低。例如某张表数据量较大，但是每天数据的变化比例很低，若对其采用每日全量同步，则会重复同步和存储大量相同的数据。
增量同步	效率高，无需同步和存储重复数据	逻辑复杂，需要将每日的新增及变化数据同原来的数据进行整合，才能使用

根据上述对比，可以得出以下结论：

通常情况，业务表数据量比较大，优先考虑增量，数据量比较小，优先考虑全量；具体选择由数仓模型决定，此处暂不详解。

下图为各表同步策略：



让天下没有难学的技术

2.2.3 数据同步工具概述

数据同步工具种类繁多,大致可分为两类,一类是以 DataX、Sqoop 为代表的基于 Select 查询的离线、批量同步工具,另一类是以 Maxwell、Canal 为代表的基于数据库数据变更日志(例如 MySQL 的 binlog, 其会实时记录所有的 insert、update 以及 delete 操作)的实时流式同步工具。

全量同步通常使用 DataX、Sqoop 等基于查询的离线同步工具。而增量同步既可以使用 DataX、Sqoop 等工具,也可使用 Maxwell、Canal 等工具,下面对增量同步不同方案进行简要对比。

增量同步方案	DataX/Sqoop	Maxwell/Canal
对数据库的要求	原理是基于查询,故若想通过 select 查询获取新增及变化数据,就要求数据表中存在 create_time、update_time 等字段,然后根据这些字段获取变更数据。	要求数据库记录变更操作,例如 MySQL 需开启 binlog。
数据的中间状态	由于是离线批量同步,故若一条数据在一天中变化多次,该方案只能获取最后一个状态,中间状态无法获取。	由于是实时获取所有的数据变更操作,所以可以获取变更数据的所有中间状态。

本项目中,全量同步采用 DataX,增量同步采用 Maxwell。

2.2.4 全量表数据同步

2.2.4.1 数据同步工具 DataX 部署

1) DataX 环境安装



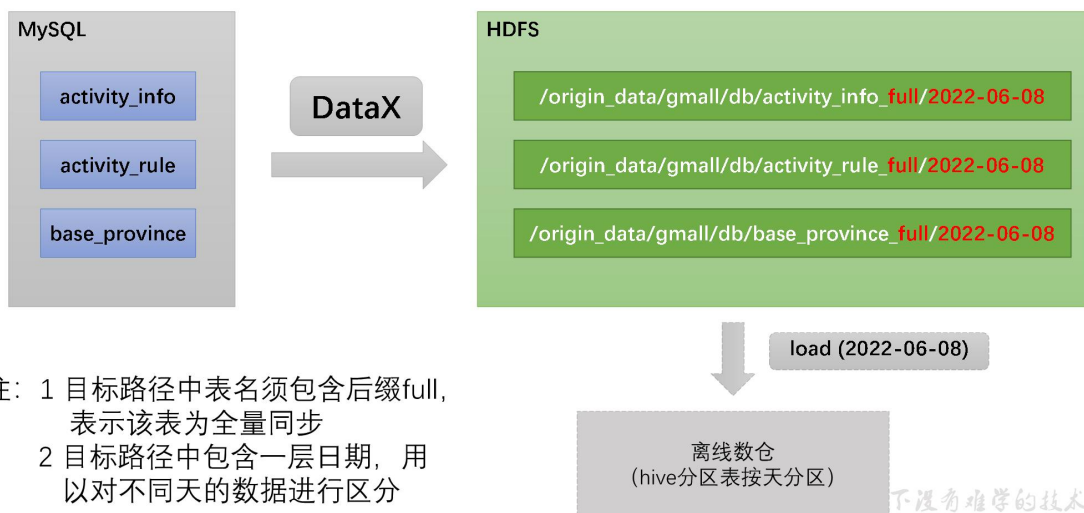
尚硅谷大数据技术
之DataX.docx

2.2.4.2 数据通道

全量表数据由 DataX 从 MySQL 业务数据库直接同步到 HDFS，具体数据流向，如下图所示。



全量表数据流向



2.2.4.3 DataX 配置文件

我们需要为每张全量表编写一个 DataX 的 json 配置文件，此处以 activity_info 为例，配置文件内容如下：

```
{
  "job": {
    "content": [
      {
        "reader": {
          "name": "mysqlreader",
          "parameter": {
            "column": [
              "id",
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：尚硅谷官网

```
        "activity_name",
        "activity_type",
        "activity_desc",
        "start_time",
        "end_time",
        "create_time"
    ],
    "connection": [
        {
            "jdbcUrl": [
                "jdbc:mysql://hadoop102:3306/gmall?useUnicode=true&allowPublicKeyRetrieval=true&characterEncoding=utf-8"
            ],
            "table": [
                "activity_info"
            ]
        }
    ],
    "password": "000000",
    "splitPk": "",
    "username": "root"
    }
},
"writer": {
    "name": "hdfswriter",
    "parameter": {
        "column": [
            {
                "name": "id",
                "type": "bigint"
            },
            {
                "name": "activity_name",
                "type": "string"
            },
            {
                "name": "activity_type",
                "type": "string"
            },
            {
                "name": "activity_desc",
                "type": "string"
            },
            {
                "name": "start_time",
                "type": "string"
            },
            {
                "name": "end_time",
                "type": "string"
            },
            {
                "name": "create_time",
                "type": "string"
            }
        ]
    }
},
```

```
        "compress": "gzip",
        "defaultFS": "hdfs://hadoop102:8020",
        "fieldDelimiter": "\t",
        "fileName": "activity_info",
        "fileType": "text",
        "path": "${targetdir}",
        "writeMode": "truncate",
        "nullFormat": ""
    }
}
    },
    "setting": {
        "speed": {
            "channel": 1
        }
    }
}
```

注：由于目标路径包含一层日期，用于对不同天的数据加以区分，故 path 参数并未写死，需在提交任务时通过参数动态传入，参数名称为 **targetdir**。

2.2.4.4 DataX 配置文件生成

1) DataX 配置文件生成器使用

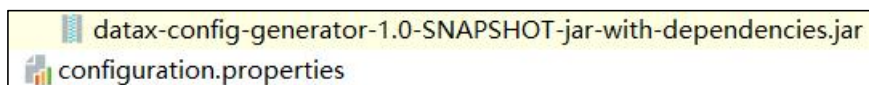


DataX配置文件生成器Java版.docx

2) 将生成器上传到服务器的/opt/module/gen_datax_config 目录

```
[atguigu@hadoop102 ~]$ mkdir /opt/module/gen_datax_config
[atguigu@hadoop102 ~]$ cd /opt/module/gen_datax_config
```

3) 上传生成器



4) 修改 configuration.properties 配置

```
mysql.username=root
mysql.password=000000
mysql.host=hadoop102
mysql.port=3306
mysql.database.import=gmall
# mysql.database.export=gmall
mysql.tables.import=activity_info,activity_rule,base_trademark,car
rt_info,base_category1,base_category2,base_category3,coupon_info,
sku_attr_value,sku_sale_attr_value,base_dic,sku_info,base_provin
ce,spu_info,base_region,promotion_pos,promotion_refer
# mysql.tables.export=
is.seperated.tables=0
```

```
hdfs.uri=hdfs://hadoop102:8020
import_out_dir=/opt/module/datax/job/import
# export_out_dir=
```

5) 执行

```
[atguigu@hadoop102 gen_datax_config]$ java -jar
datax-config-generator-1.0-SNAPSHOT-jar-with-dependencies.jar
```

6) 观察结果

```
[atguigu@hadoop102 ~]$ ll /opt/module/datax/job/import
总用量 68
-rw-rw-r--. 1 atguigu atguigu 976 2月 3 21:56 activity_info.json
-rw-rw-r--. 1 atguigu atguigu 1116 2月 3 21:56 activity_rule.json
-rw-rw-r--. 1 atguigu atguigu 747 2月 3 21:56 base_category1.json
-rw-rw-r--. 1 atguigu atguigu 802 2月 3 21:56 base_category2.json
-rw-rw-r--. 1 atguigu atguigu 802 2月 3 21:56 base_category3.json
-rw-rw-r--. 1 atguigu atguigu 814 2月 3 21:56 base_dic.json
-rw-rw-r--. 1 atguigu atguigu 942 2月 3 21:56 base_province.json
-rw-rw-r--. 1 atguigu atguigu 758 2月 3 21:58 base_region.json
-rw-rw-r--. 1 atguigu atguigu 800 2月 3 21:56 base_trademark.json
-rw-rw-r--. 1 atguigu atguigu 1234 2月 3 21:56 cart_info.json
-rw-rw-r--. 1 atguigu atguigu 1461 2月 3 21:56 coupon_info.json
-rw-rw-r--. 1 atguigu atguigu 868 2月 3 21:56 promotion_pos.json
-rw-rw-r--. 1 atguigu atguigu 760 2月 3 21:56 promotion_refer.json
-rw-rw-r--. 1 atguigu atguigu 943 2月 3 21:56 sku_attr_value.json
-rw-rw-r--. 1 atguigu atguigu 1125 2月 3 21:56 sku_info.json
-rw-rw-r--. 1 atguigu atguigu 1051 2月 3 21:56 sku_sale_attr_value.json
-rw-rw-r--. 1 atguigu atguigu 898 2月 3 21:56 spu_info.json
```

2.2.4.5 测试生成的 DataX 配置文件

以 activity_info 为例，测试用脚本生成的配置文件是否可用。

1) 创建目标路径

由于 DataX 同步任务要求目标路径提前存在，故需手动创建路径，当前 activity_info 表的目标路径应为/origin_data/gmall/db/activity_info_full/2022-06-08。

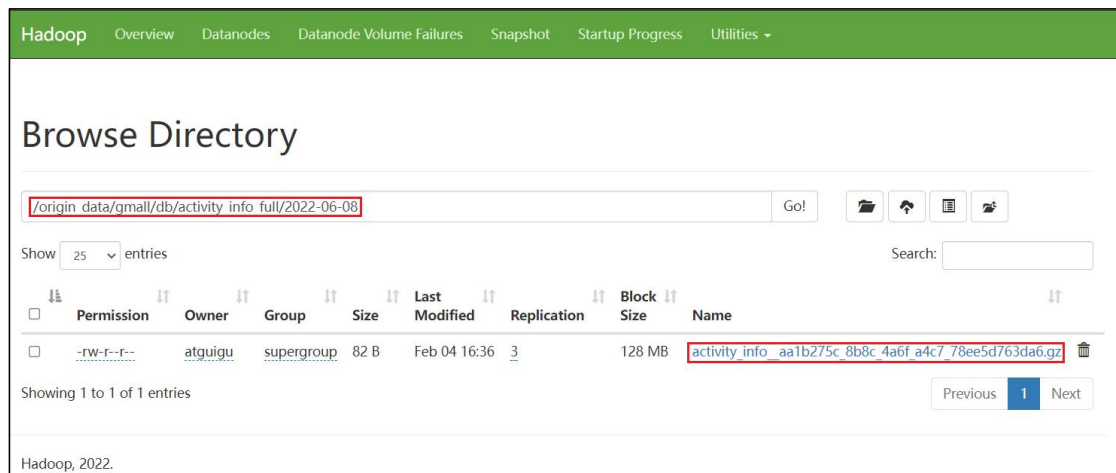
```
[atguigu@hadoop102 bin]$ hadoop fs -mkdir -p
/origin_data/gmall/db/activity_info_full/2022-06-08
```

2) 执行 DataX 同步命令

```
[atguigu@hadoop102 bin]$ python /opt/module/datax/bin/datax.py
-p"-Dtargetdir=/origin_data/gmall/db/activity_info_full/2022-06-08"
/opt/module/datax/job/import/gmall.activity_info.json
```

3) 观察同步结果

观察 HDFS 目标路径是否出现数据。



2.2.4.6 全量表数据同步脚本

为方便使用以及后续的任务调度，此处编写一个全量表数据同步脚本。

1) 在 `~/bin` 目录创建 `mysql_to_hdfs_full.sh`

```
[atguigu@hadoop102 bin]$ vim ~/bin/mysql_to_hdfs_full.sh
```

脚本内容如下

```
#!/bin/bash

DATAX_HOME=/opt/module/datax

# 如果传入日期则 do_date 等于传入的日期，否则等于前一天日期
if [ -n "$2" ] ;then
    do_date=$2
else
    do_date=`date -d "-1 day" +%F`
fi

#处理目标路径，此处的处理逻辑是，如果目标路径不存在，则创建；若存在，则清空，目的是保证同步任务可重复执行
handle_targetdir() {
    hadoop fs -test -e $1
    if [[ $? -eq 1 ]]; then
        echo "路径$1 不存在，正在创建....."
        hadoop fs -mkdir -p $1
    else
        echo "路径$1 已经存在"
    fi
}

#数据同步
import_data() {
    datax_config=$1
    target_dir=$2

    handle_targetdir $target_dir
    python $DATAX_HOME/bin/datax.py -p"-Dtargetdir=$target_dir"
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：尚硅谷官网


```
$datax_config
}

case $1 in
"activity_info")
    import_data
/opt/module/datax/job/import/gmall.activity_info.json
/origin_data/gmall/db/activity_info_full/$do_date
;;
"activity_rule")
    import_data
/opt/module/datax/job/import/gmall.activity_rule.json
/origin_data/gmall/db/activity_rule_full/$do_date
;;
"base_category1")
    import_data
/opt/module/datax/job/import/gmall.base_category1.json
/origin_data/gmall/db/base_category1_full/$do_date
;;
"base_category2")
    import_data
/opt/module/datax/job/import/gmall.base_category2.json
/origin_data/gmall/db/base_category2_full/$do_date
;;
"base_category3")
    import_data
/opt/module/datax/job/import/gmall.base_category3.json
/origin_data/gmall/db/base_category3_full/$do_date
;;
"base_dic")
    import_data /opt/module/datax/job/import/gmall.base_dic.json
/origin_data/gmall/db/base_dic_full/$do_date
;;
"base_province")
    import_data
/opt/module/datax/job/import/gmall.base_province.json
/origin_data/gmall/db/base_province_full/$do_date
;;
"base_region")
    import_data /opt/module/datax/job/import/gmall.base_region.json
/origin_data/gmall/db/base_region_full/$do_date
;;
"base_trademark")
    import_data
/opt/module/datax/job/import/gmall.base_trademark.json
/origin_data/gmall/db/base_trademark_full/$do_date
;;
"cart_info")
    import_data /opt/module/datax/job/import/gmall.cart_info.json
/origin_data/gmall/db/cart_info_full/$do_date
;;
"coupon_info")
    import_data /opt/module/datax/job/import/gmall.coupon_info.json
/origin_data/gmall/db/coupon_info_full/$do_date
;;
"sku_attr_value")
    import_data
```

```
/opt/module/datax/job/import/gmall.skus_attr_value.json
/origin_data/gmall/db/skus_attr_value_full/$do_date
;;
"skus_info")
import_data /opt/module/datax/job/import/gmall.skus_info.json
/origin_data/gmall/db/skus_info_full/$do_date
;;
"skus_sale_attr_value")
import_data
/opt/module/datax/job/import/gmall.skus_sale_attr_value.json
/origin_data/gmall/db/skus_sale_attr_value_full/$do_date
;;
"spu_info")
import_data /opt/module/datax/job/import/gmall.spu_info.json
/origin_data/gmall/db/spu_info_full/$do_date
;;
"promotion_pos")
import_data
/opt/module/datax/job/import/gmall.promotion_pos.json
/origin_data/gmall/db/promotion_pos_full/$do_date
;;
"promotion_refer")
import_data
/opt/module/datax/job/import/gmall.promotion_refer.json
/origin_data/gmall/db/promotion_refer_full/$do_date
;;
"all")
import_data
/opt/module/datax/job/import/gmall.activity_info.json
/origin_data/gmall/db/activity_info_full/$do_date
import_data
/opt/module/datax/job/import/gmall.activity_rule.json
/origin_data/gmall/db/activity_rule_full/$do_date
import_data
/opt/module/datax/job/import/gmall.base_category1.json
/origin_data/gmall/db/base_category1_full/$do_date
import_data
/opt/module/datax/job/import/gmall.base_category2.json
/origin_data/gmall/db/base_category2_full/$do_date
import_data
/opt/module/datax/job/import/gmall.base_category3.json
/origin_data/gmall/db/base_category3_full/$do_date
import_data /opt/module/datax/job/import/gmall.base_dic.json
/origin_data/gmall/db/base_dic_full/$do_date
import_data
/opt/module/datax/job/import/gmall.base_province.json
/origin_data/gmall/db/base_province_full/$do_date
import_data /opt/module/datax/job/import/gmall.base_region.json
/origin_data/gmall/db/base_region_full/$do_date
import_data
/opt/module/datax/job/import/gmall.base_trademark.json
/origin_data/gmall/db/base_trademark_full/$do_date
import_data /opt/module/datax/job/import/gmall.cart_info.json
/origin_data/gmall/db/cart_info_full/$do_date
import_data /opt/module/datax/job/import/gmall.coupon_info.json
/origin_data/gmall/db/coupon_info_full/$do_date
import_data
```

```

/opt/module/datax/job/import/gmall.skus_attr_value.json
/origin_data/gmall/db/skus_attr_value_full/$do_date
import_data /opt/module/datax/job/import/gmall.skus_info.json
/origin_data/gmall/db/skus_info_full/$do_date
import_data
/opt/module/datax/job/import/gmall.skus_sale_attr_value.json
/origin_data/gmall/db/skus_sale_attr_value_full/$do_date
import_data /opt/module/datax/job/import/gmall.spu_info.json
/origin_data/gmall/db/spu_info_full/$do_date
import_data
/opt/module/datax/job/import/gmall.promotion_pos.json
/origin_data/gmall/db/promotion_pos_full/$do_date
import_data
/opt/module/datax/job/import/gmall.promotion_refer.json
/origin_data/gmall/db/promotion_refer_full/$do_date
;;
esac

```

2) 为 mysql_to_hdfs_full.sh 增加执行权限

```
[atguigu@hadoop102 bin]$ chmod 777 ~/bin/mysql_to_hdfs_full.sh
```

3) 测试同步脚本

```
[atguigu@hadoop102 bin]$ mysql_to_hdfs_full.sh all 2022-06-08
```

4) 检查同步结果

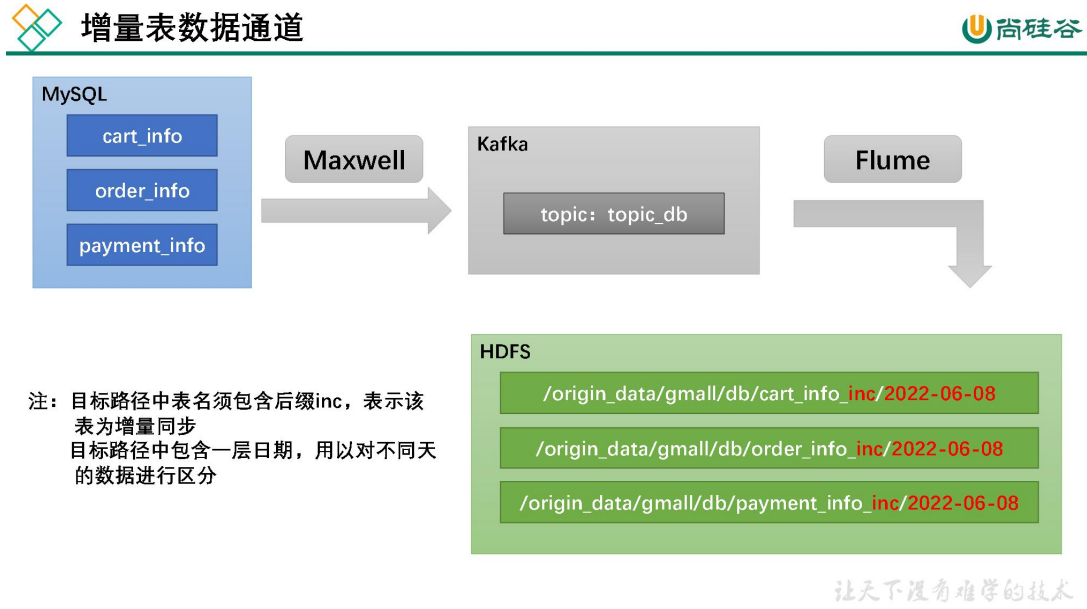
查看 HDFS 目录路径是否出现全量表数据，全量表共 17 张。

Permission	Owner	Group	Size	Last Modified	Replication	Block Size	Name
drwxr-xr-x	atguigu	supergroup	0 B	Feb 04 16:39	0	0 B	activity_info_full
drwxr-xr-x	atguigu	supergroup	0 B	Feb 04 16:40	0	0 B	activity_rule_full
drwxr-xr-x	atguigu	supergroup	0 B	Feb 04 16:40	0	0 B	base_category1_full
drwxr-xr-x	atguigu	supergroup	0 B	Feb 04 16:40	0	0 B	base_category2_full
drwxr-xr-x	atguigu	supergroup	0 B	Feb 04 16:40	0	0 B	base_category3_full
drwxr-xr-x	atguigu	supergroup	0 B	Feb 04 16:41	0	0 B	base_dic_full
drwxr-xr-x	atguigu	supergroup	0 B	Feb 04 16:41	0	0 B	base_province_full
drwxr-xr-x	atguigu	supergroup	0 B	Feb 04 16:41	0	0 B	base_region_full
drwxr-xr-x	atguigu	supergroup	0 B	Feb 04 16:41	0	0 B	base_trademark_full
drwxr-xr-x	atguigu	supergroup	0 B	Feb 04 16:42	0	0 B	cart_info_full
drwxr-xr-x	atguigu	supergroup	0 B	Feb 04 16:42	0	0 B	coupon_info_full
drwxr-xr-x	atguigu	supergroup	0 B	Feb 04 16:43	0	0 B	promotion_pos_full
drwxr-xr-x	atguigu	supergroup	0 B	Feb 04 16:49	0	0 B	promotion_refer_full
drwxr-xr-x	atguigu	supergroup	0 B	Feb 04 16:42	0	0 B	sku_attr_value_full
drwxr-xr-x	atguigu	supergroup	0 B	Feb 04 16:42	0	0 B	sku_info_full
drwxr-xr-x	atguigu	supergroup	0 B	Feb 04 16:43	0	0 B	sku_sale_attr_value_full
drwxr-xr-x	atguigu	supergroup	0 B	Feb 04 16:51	0	0 B	spu_info_full

Showing 1 to 17 of 17 entries

2.2.5 增量表数据同步

2.2.5.1 数据通道



2.2.5.2 Flume 配置

1) Flume 配置概述

Flume 需要将 Kafka 中 topic_db 主题的数据传输到 HDFS，故其需选用 KafkaSource 以及 HDFSSink，Channel 选用 FileChannel。

需要注意的是，HDFSSink 需要将不同 MySQL 业务表的数据写到不同的路径，并且路径中应当包含一层日期，用于区分每天的数据。关键配置如下：

KafkaSource

```
#订阅业务数据的topic
kafka.topics =topic_db

#自定义时间戳拦截器为Event增加一个header, key
#为timestamp, value为json字符串中ts字段的值
interceptors = i1
interceptors.i1.type=TimestampAndTableNameInte
rceptor$Builder
```

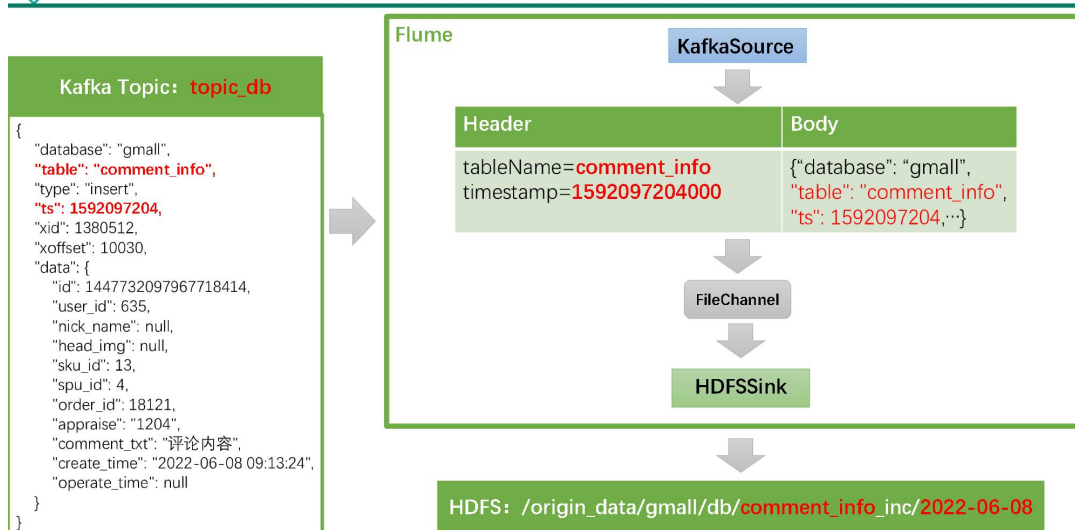
FileChannel

HDFS Sink

```
#path中包含自定义转义序列和时间转移序列, 用于
#将不同topic的数据放到不同的路径, 以及不同
#日期的数据放到不同的路径
path=/origin_data/gmall/db/{tableName}_inc/%Y-
%m-%d
```

具体数据示例如下:

Flume数据示例



让天下没有难学的技术

2) Flume 配置实操

(1) 创建 Flume 配置文件

在 hadoop104 节点的 Flume 的 job 目录下创建 kafka_to_hdfs_db.conf

```
[atguigu@hadoop104 flume]$ vim job/kafka_to_hdfs_db.conf
```

(2) 配置文件内容如下

```
a1.sources = r1
a1.channels = c1
```

更多 Java - 大数据 - 前端 - python 人工智能资料下载, 可百度访问: 尚硅谷官网

```
a1.sinks = k1

a1.sources.r1.type = org.apache.flume.source.kafka.KafkaSource
a1.sources.r1.batchSize = 5000
a1.sources.r1.batchDurationMillis = 2000
a1.sources.r1.kafka.bootstrap.servers =
hadoop102:9092,hadoop103:9092
a1.sources.r1.kafka.topics = topic_db
a1.sources.r1.kafka.consumer.group.id = flume
a1.sources.r1.setTopicHeader = true
a1.sources.r1.topicHeader = topic
a1.sources.r1.interceptors = i1
a1.sources.r1.interceptors.i1.type =
com.atguigu.gmall.flume.interceptor.TimestampAndTableNameIntercep
tor$Builder

a1.channels.c1.type = file
a1.channels.c1.checkpointDir =
/opt/module/flume-1.10/checkpoint/behavior2
a1.channels.c1.dataDirs = /opt/module/flume-1.10/data/behavior2/
a1.channels.c1.maxFileSize = 2146435071
a1.channels.c1.capacity = 1000000
a1.channels.c1.keep-alive = 6

## sink1
a1.sinks.k1.type = hdfs
a1.sinks.k1.hdfs.path =
/origin_data/gmall/db/{tableName}_inc/%Y-%m-%d
a1.sinks.k1.hdfs.filePrefix = db
a1.sinks.k1.hdfs.round = false

a1.sinks.k1.hdfs.rollInterval = 10
a1.sinks.k1.hdfs.rollSize = 134217728
a1.sinks.k1.hdfs.rollCount = 0

a1.sinks.k1.hdfs.fileType = CompressedStream
a1.sinks.k1.hdfs.codeC = gzip

## 拼装
a1.sources.r1.channels = c1
a1.sinks.k1.channel= c1
```

(3) 编写 Flume 拦截器

在 com.atguigu.gmall.flume.interceptor 包下创建 TimestampAndTableNameInterceptor 类。

```
package com.atguigu.gmall.flume.interceptor;

import com.alibaba.fastjson.JSONObject;
import org.apache.flume.Context;
import org.apache.flume.Event;
import org.apache.flume.interceptor.Interceptor;

import java.nio.charset.StandardCharsets;
import java.util.List;
import java.util.Map;
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载, 可百度访问: [尚硅谷官网](#)

```
public class TimestampAndTableNameInterceptor implements Interceptor
{
    @Override
    public void initialize() {

    }

    @Override
    public Event intercept(Event event) {

        Map<String, String> headers = event.getHeaders();
        String log = new String(event.getBody(),
StandardCharsets.UTF_8);

        JSONObject jsonObject = JSONObject.parseObject(log);

        Long ts = jsonObject.getLong("ts");
        //Maxwell 输出的数据中的 ts 字段时间戳单位为秒, Flume HDFSSink 要求单
        位为毫秒
        String timeMills = String.valueOf(ts * 1000);

        String tableName = jsonObject.getString("table");

        headers.put("timestamp", timeMills);
        headers.put("tableName", tableName);
        return event;

    }

    @Override
    public List<Event> intercept(List<Event> events) {

        for (Event event : events) {
            intercept(event);
        }

        return events;
    }

    @Override
    public void close() {

    }

    public static class Builder implements Interceptor.Builder {

        @Override
        public Interceptor build() {
            return new TimestampAndTableNameInterceptor ();
        }

        @Override
        public void configure(Context context) {

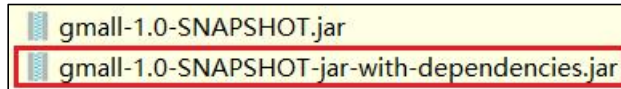
        }

    }
}
```



```
}
}
```

(3) 重新打包



(4) 删除 hadoop104 的 /opt/module/flume/lib 目录下的 gmall-1.0-SNAPSHOT-jar-with-dependencies.jar 文件

```
[atguigu@hadoop104 lib]$ cd /opt/module/flume/lib/
[atguigu@hadoop104 lib]$ rm gmall-1.0-SNAPSHOT-jar-with-dependencies.jar
```

(5) 将打好的包放入到 hadoop104 的 /opt/module/flume/lib 文件夹下

```
[atguigu@hadoop104 lib]$ ls | grep gmall
gmall-1.0-SNAPSHOT-jar-with-dependencies.jar
```

3) 通道测试

(1) 启动 Zookeeper、Kafka 集群及 Maxwell

(2) 启动 hadoop104 的 Flume

```
[atguigu@hadoop104 flume]$ bin/flume-ng agent -n a1 -c conf/ -f
job/kafka_to_hdfs_db.conf
```

(3) 生成模拟数据

确保 Maxwell 正在运行，而后生成数据。

```
[atguigu@hadoop102 bin]$ lg.sh
```

(4) 观察 HDFS 目标路径

Hadoop

Overview

Datanodes

Datanode Volume Failures

Snapshot

Startup Progress

Utilities

Browse Directory

/origin_data/gmall/db

Go!

Show

25

entries

Search:

☐	Permission	Owner	Group	Size	Last Modified	Replication	Block Size	Name	
☐	drwxr-xr-x	atguigu	supergroup	0 B	Feb 04 16:39	0	0 B	activity_info_full	
☐	drwxr-xr-x	atguigu	supergroup	0 B	Feb 04 16:40	0	0 B	activity_rule_full	
☐	drwxr-xr-x	atguigu	supergroup	0 B	Feb 04 16:40	0	0 B	base_category1_full	
☐	drwxr-xr-x	atguigu	supergroup	0 B	Feb 04 16:40	0	0 B	base_category2_full	
☐	drwxr-xr-x	atguigu	supergroup	0 B	Feb 04 16:40	0	0 B	base_category3_full	
☐	drwxr-xr-x	atguigu	supergroup	0 B	Feb 04 16:41	0	0 B	base_dic_full	
☐	drwxr-xr-x	atguigu	supergroup	0 B	Feb 04 16:41	0	0 B	base_province_full	
☐	drwxr-xr-x	atguigu	supergroup	0 B	Feb 04 16:41	0	0 B	base_region_full	
☐	drwxr-xr-x	atguigu	supergroup	0 B	Feb 04 16:41	0	0 B	base_trademark_full	
☐	drwxr-xr-x	atguigu	supergroup	0 B	Feb 04 16:42	0	0 B	cart_info_full	
☐	drwxr-xr-x	atguigu	supergroup	0 B	Feb 04 17:15	0	0 B	cart_info_inc	

更多 Java - 大数据 - 前端 - python 人工智能资料下载，可百度访问：尚硅谷官网

增量表目录出现，数据通道已打通。

(5) 数据目标路径的日期说明

仔细观察，会发现目标路径中的日期，并非模拟数据的业务日期，而是当前日期。这是由于 Maxwell 输出的 JSON 字符串中的 ts 字段的值，是数据的变动日期。而真实场景下，数据的业务日期与变动日期应当是一致的。教学环境下需要修改时间戳日期，下文详述。

4) 编写 Flume 启停脚本

为方便使用，此处编写一个 Flume 的启停脚本。

(1) 在 hadoop102 节点的/home/atguigu/bin 目录下创建脚本 f3.sh

```
[atguigu@hadoop102 bin]$ vim f3.sh
```

在脚本中填写如下内容。

```
#!/bin/bash

case $1 in
"start")
    echo " -----启动 hadoop104 业务数据 flume-----"
    ssh hadoop104 "nohup /opt/module/flume-1.10.1/bin/flume-ng
agent -n al -c /opt/module/flume-1.10.1/conf -f
/opt/module/flume-1.10.1/kafkatohdfs/kafka_to_hdfs_db.conf >/dev/n
ull 2>&1 &"
    ;;
"stop")
    echo " -----停止 hadoop104 业务数据 flume-----"
    ssh hadoop104 "ps -ef | grep kafka_to_hdfs_db | grep -v grep
|awk '{print \$2}' | xargs -n1 kill"
    ;;
esac
```

(2) 增加脚本执行权限

```
[atguigu@hadoop102 bin]$ chmod 777 f3.sh
```

(3) f3 启动

```
[atguigu@hadoop102 module]$ f3.sh start
```

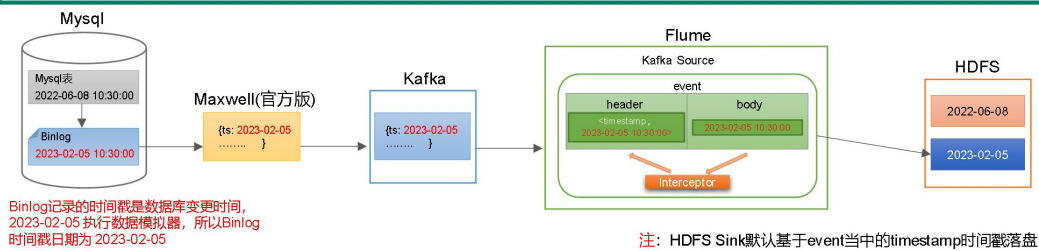
(4) f3 停止

```
[atguigu@hadoop102 module]$ f3.sh stop
```

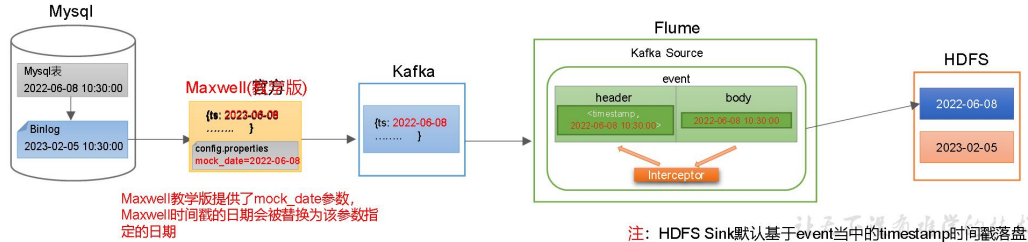
2.2.5.3 Maxwell 配置

1) Maxwell 时间戳问题

Maxwell时间戳问题



如何解决？



为了让 Maxwell 时间戳日期与模拟的业务日期保持一致，对 Maxwell 源码进行改动，增加了 mock_date 参数，在/opt/module/maxwell/config.properties 文件中将该参数的值修改为业务日期即可。

2) 补充 mock.date 参数

配置参数如下。

```
log_level=info

#Maxwell 数据发送目的地，可选配置有
stdout|file|kafka|kinesis|pubsub|sqs|rabbitmq|redis
producer=kafka
# 目标 Kafka 集群地址
kafka.bootstrap.servers=hadoop102:9092,hadoop103:9092,hadoop104:9092
# 目标 Kafka topic，可静态配置，例如:maxwell，也可动态配置，例如:
#{database}_#{table}
kafka_topic=topic_db

# MySQL 相关配置
host=hadoop102
user=maxwell
password=maxwell
jdbc_options=useSSL=false&serverTimezone=Asia/Shanghai&allowPublicKeyRetrieval=true

# 过滤 gmall 中的 z_log 表数据，该表是日志数据的备份，无须采集
filter=exclude:gmall.z_log
# 指定数据按照主键分组进入 Kafka 不同分区，避免数据倾斜
producer_partition_by=primary_key

# 修改数据时间戳的日期部分
mock_date=2022-06-08
```

3) 重新启动 Maxwell

```
[atguigu@hadoop102 bin]$ mxw.sh restart
```

4) 重新生成模拟数据

```
[atguigu@hadoop102 bin]$ lg.sh
```

5) 观察 HDFS 目标路径日期是否与业务日期保持一致

2.2.5.4 增量表首日全量同步

通常情况下，增量表需要在首日进行一次全量同步，后续每日再进行增量同步，首日全量同步可以使用 Maxwell 的 bootstrap 功能，方便起见，下面编写一个增量表首日全量同步脚本。

1) 在~/bin 目录创建 mysql_to_kafka_inc_init.sh

```
[atguigu@hadoop102 bin]$ vim mysql_to_kafka_inc_init.sh
```

脚本内容如下

```
#!/bin/bash

# 该脚本的作用是初始化所有的增量表，只需执行一次

MAXWELL_HOME=/opt/module/maxwell-1.29.2

import_data() {
    $MAXWELL_HOME/bin/maxwell-bootstrap --database gmall --table $1
    --config $MAXWELL_HOME/config.properties
}

case $1 in
    "cart_info")
        import_data cart_info
        ;;
    "comment_info")
        import_data comment_info
        ;;
    "coupon_use")
        import_data coupon_use
        ;;
    "favor_info")
        import_data favor_info
        ;;
    "order_detail")
        import_data order_detail
        ;;
    "order_detail_activity")
        import_data order_detail_activity
        ;;
    "order_detail_coupon")
        import_data order_detail_coupon
        ;;
    "order_info")
        import_data order_info
        ;;
    "order_refund_info")
```

```
import_data order_refund_info
;;
"order_status_log")
import_data order_status_log
;;
"payment_info")
import_data payment_info
;;
"refund_payment")
import_data refund_payment
;;
"user_info")
import_data user_info
;;
"all")
import_data cart_info
import_data comment_info
import_data coupon_use
import_data favor_info
import_data order_detail
import_data order_detail_activity
import_data order_detail_coupon
import_data order_info
import_data order_refund_info
import_data order_status_log
import_data payment_info
import_data refund_payment
import_data user_info
;;
esac
```

2) 为 mysql_to_kafka_inc_init.sh 增加执行权限

```
[atguigu@hadoop102 bin]$ chmod 777 ~/bin/mysql_to_kafka_inc_init.sh
```

3) 测试同步脚本

(1) 清理历史数据

为方便查看结果，现将 HDFS 上之前同步的增量表数据删除。

```
[atguigu@hadoop102 ~]$ hadoop fs -ls /origin_data/gmall/db | grep _inc | awk '{print $8}' | xargs hadoop fs -rm -r -f
```

(2) 执行同步脚本

```
[atguigu@hadoop102 bin]$ mysql_to_kafka_inc_init.sh all
```

4) 检查同步结果

观察 HDFS 上是否重新出现增量表数据。

2.3 采集通道启动/停止脚本

1) 在/home/atguigu/bin 目录下创建脚本 cluster.sh

```
[atguigu@hadoop102 bin]$ vim cluster.sh
```

在脚本中填写如下内容。

```
#!/bin/bash
case $1 in
```

```
"start"){
    echo ===== 启动 集群 =====

    #启动 Zookeeper 集群
    zk.sh start

    #启动 Hadoop 集群
    myhadoop.sh start

    #启动 Kafka 采集集群
    kf.sh start

    #启动采集 Flume
    f102.sh start

#启动日志消费 Flume
    f104_log.sh start

#启动业务消费 Flume
    f104_db.sh start

#启动 maxwell
    mxw.sh start

    };;
"stop"){
    echo ===== 停止 集群 =====

#停止 Maxwell
    mxw.sh stop

#停止 业务消费 Flume
    f104_db.sh stop

#停止 日志消费 Flume
    f104_log.sh stop

#停止 日志采集 Flume
    f102.sh stop

    #停止 Kafka 采集集群
    kf.sh stop

    #停止 Hadoop 集群
    myhadoop.sh stop

#循环直至 Kafka 集群进程全部停止
    kafka_count=$(xcall jps | grep Kafka | wc -l)
    while [ $kafka_count -gt 0 ]
    do
        sleep 1
        kafka_count=$(jpsall | grep Kafka | wc -l)
        echo "当前未停止的 Kafka 进程数为 $kafka_count"
    done
```

```
#停止 Zookeeper 集群
zk.sh stop

};;
esac
```

2) 增加脚本执行权限

```
[atguigu@hadoop102 bin]$ chmod 777 cluster.sh
```

3) cluster 集群启动脚本

```
[atguigu@hadoop102 module]$ cluster.sh start
```

4) cluster 集群停止脚本

```
[atguigu@hadoop102 module]$ cluster.sh stop
```

第 3 章 数仓环境准备

4.1 Hive 安装部署

1) 把 hive-3.1.3.tar.gz 上传到 linux 的/opt/software 目录下

2) 解压 hive-3.1.3.tar.gz 到/opt/module/目录下

```
[atguigu@hadoop102 software]$ tar -zxvf /opt/software/hive-3.1.3.tar.gz -C /opt/module/
```

3) 修改 hive-3.1.3-bin.tar.gz 的名称为 hive

```
[atguigu@hadoop102 software]$ mv /opt/module/apache-hive-3.1.3-bin/ /opt/module/hive
```

4) 修改/etc/profile.d/my_env.sh, 添加环境变量

```
[atguigu@hadoop102 software]$ sudo vim /etc/profile.d/my_env.sh
```

添加内容

```
#HIVE_HOME
export HIVE_HOME=/opt/module/hive
export PATH=$PATH:$HIVE_HOME/bin
```

重启 Xshell 对话框或者 source 一下 /etc/profile.d/my_env.sh 文件, 使环境变量生效。

```
[atguigu@hadoop102 software]$ source /etc/profile.d/my_env.sh
```

5) 解决日志 Jar 包冲突, 进入/opt/module/hive/lib 目录

```
[atguigu@hadoop102 lib]$ mv log4j-slf4j-impl-2.17.1.jar log4j-slf4j-impl-2.17.1.jar.bak
```

4.2 Hive 元数据配置到 MySQL

4.2.1 拷贝驱动

将 MySQL 的 JDBC 驱动拷贝到 Hive 的 lib 目录下。

```
[atguigu@hadoop102 lib]$ cp /opt/software/mysql/mysql-connector-j-8.0.31.jar /opt/module/hive/lib/
```

4.2.2 配置 Metastore 到 MySQL

在\$HIVE_HOME/conf 目录下新建 hive-site.xml 文件。

```
[atguigu@hadoop102 conf]$ vim hive-site.xml
```

添加如下内容。

```
<?xml version="1.0"?>
<?xml-stylesheet type="text/xsl" href="configuration.xsl"?>
<configuration>
  <!--配置 Hive 保存元数据信息所需的 MySQL URL 地址-->
  <property>
    <name>javax.jdo.option.ConnectionURL</name>

<value>jdbc:mysql://hadoop102:3306/metastore?useSSL=false&use
Unicode=true&characterEncoding=UTF-8&allowPublicKeyRetrie
val=true</value>
  </property>

  <!--配置 Hive 连接 MySQL 的驱动全类名-->
  <property>
    <name>javax.jdo.option.ConnectionDriverName</name>
    <value>com.mysql.cj.jdbc.Driver</value>
  </property>

  <!--配置 Hive 连接 MySQL 的用户名 -->
  <property>
    <name>javax.jdo.option.ConnectionUserName</name>
    <value>root</value>
  </property>

  <!--配置 Hive 连接 MySQL 的密码 -->
  <property>
    <name>javax.jdo.option.ConnectionPassword</name>
    <value>000000</value>
  </property>

  <property>
    <name>hive.metastore.warehouse.dir</name>
    <value>/user/hive/warehouse</value>
  </property>

  <property>
    <name>hive.metastore.schema.validation</name>
    <value>false</value>
  </property>

  <property>
    <name>hive.server2.thrift.port</name>
    <value>10000</value>
  </property>

  <property>
    <name>hive.server2.thrift.bind.host</name>
    <value>hadoop102</value>
  </property>
```

更多 Java - 大数据 - 前端 - python 人工智能资料下载，可百度访问：尚硅谷官网

```
<property>
  <name>hive.metastore.event.db.notification.api.auth</name>
  <value>false</value>
</property>

<property>
  <name>hive.cli.print.header</name>
  <value>true</value>
</property>

<property>
  <name>hive.cli.print.current.db</name>
  <value>true</value>
</property>
</configuration>
```

4.3 启动 Hive

4.3.1 初始化元数据库

1) 登陆 MySQL

```
[atguigu@hadoop102 conf]$ mysql -uroot -p000000
```

2) 新建 Hive 元数据库

```
mysql> create database metastore;
```

3) 初始化 Hive 元数据库

```
[atguigu@hadoop102 bin]$ schematool -initSchema -dbType mysql
-verbose
```

4) 修改元数据库字符集

Hive 元数据库的字符集默认为 Latin1，由于其不支持中文字符，所以建表语句中如果包含中文注释，会出现乱码现象。如需解决乱码问题，须做以下修改。

修改 Hive 元数据库中存储注释的字段的字符集为 utf-8。

(1) 字段注释

```
mysql> use metastore;
mysql> alter table COLUMNS_V2 modify column COMMENT varchar(256)
character set utf8;
```

(2) 表注释

```
mysql> alter table TABLE_PARAMS modify column PARAM_VALUE mediumtext
character set utf8;
```

4) 退出 mysql

```
mysql> quit;
```

4.3.2 启动 Hive 客户端

1) 启动 Hive 客户端

```
[atguigu@hadoop102 hive]$ hive
```


2) 查看一下数据库

```
hive (default)> show databases;  
OK  
database_name  
default  
Time taken: 0.955 seconds, Fetched: 1 row(s)
```